Machine Learning for Molecular Engineering
3/7/10/20.01 (U) 3/7/10/20.C51 (G)
Spring 2025

Problem Set #5

Date: April 26, 2025
Due: May 5, 2025 @ 11:59 pm

Instructions

- This problem set has two modeling tasks with several sub-questions. Some are marked grad version, which are required for graduate students (X.C51) but optional for others. Points for all students are in blue, while grad-only points are in orange. The total points are 75 for undergraduates and 100 for graduates.
- To get started, open your Google Colab or Jupyter notebook starting from the problem set template file `pset5.ipynb` (direct Colab link). You will need to use the data files here and here. Make your own copy of this template to save changes, by selecting *Save a copy in Drive*. If you have not used Google Colab, you might find this example notebook helpful.
- **Important:** This problem set requires a GPU. Before you start in Google Colab, find *Notebook Settings* under the *Edit* menu. In the *Hardware accelerator* drop-down, select a GPU as your hardware accelerator. Changing the runtime resets the notebook, so make this change before starting!
- Collaboration is encouraged and AI tools are permitted, but submitting work that is not your own is plagiarism. Any collaboration or assistance from an LLM (including utilities present within Google Colab) should be described at the end of your submission.
- Upon completion, submit your IPython Notebook `pset5.ipynb` to Gradescope. Ensure your submission includes *all* necessary code to be run without error, with all plots and outputs. Comments are encouraged; put answers to conceptual questions in Markdown/Text cells.

Background

SMILES for representing molecules

Whether it's designing specific ligands for enzyme inhibition, predicting the biological activity of small molecules, or exploring chemical space for drug discovery, understanding how to encode chemical structures as strings, called SMILES, is essential for computational biology (as well as computational chemistry, of course). The simplified molecular-input line-entry system (SMILES) is a text-based notation describing the structure of molecules using short ASCII strings. In terms of a graph-based computational procedure, SMILES strings are generated by printing the symbol nodes encountered in a breadth-first traversal of the graphs, typically excluding hydrogen atoms. Any cycles are broken so that the graph becomes an acyclic (tree-structured) graph. Numbers indicate connections between non-adjacent characters in the SMILES string; parentheses are used to indicate points of branching on the tree. Every molecule can be represented by multiple SMILES strings and there exist algorithms that can reproducibly generate the *canonical* SMILES string for a molecule.

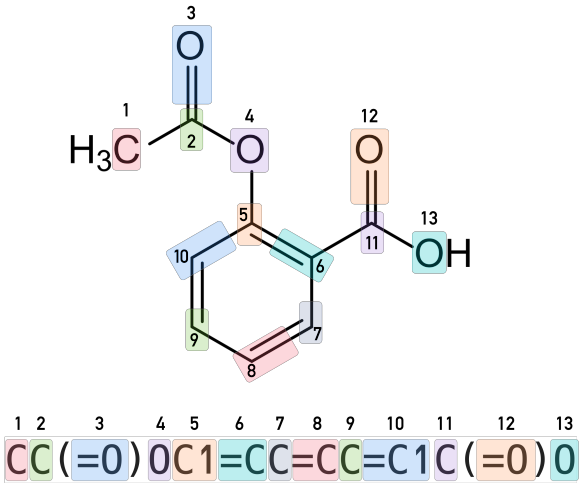

Figure 1: A molecular structure can be represented by SMILES string (source)

Part 1: Variational auto-encoders for SMILES strings

Variational auto-encoders (VAEs) are a class of generative models. Adapted from the VAE architecture for SMILES detailed in Gómez-Bombarelli et al. (2018) [1], this problem similarly implements a VAE for SMILES strings. The encoder for this problem consists of gated recurrent units (GRUs) to encode a SMILES string into a latent representation. The decoder is also a stacked GRU that takes the latent representation to reconstruct the input SMILES. The autoencoder is trained to use the encoder and decoder to reconstructs your input as closely as possible. You will be asked to implement the sampling and loss function for a SMILES-VAE. Because training a VAE on a large dataset can be computationally demanding, we have provided a SMILES-VAE model that is pre-trained on 1 million molecules. You can load the model with the code we provide. You will train on a smaller dataset of of 50,000 molecules to fine-tune the model. It is still a lot of data, so please train your model on a GPU.

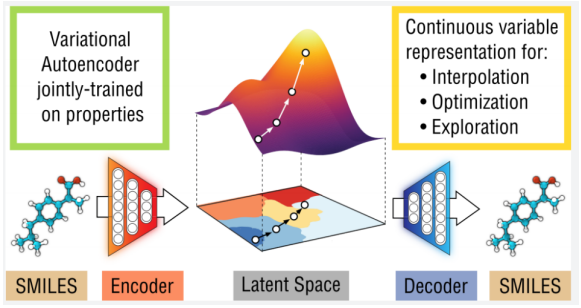

Figure 2: Applying a VAE on SMILES strings for molecular design from [1]

Part 1.1 (5 points) Encode SMILES strings into numerical vectors

As introduced in the background section, molecules can be represented as 1D strings with SMILES representation. We have provided a list of string characters in `moses_charset` which you can use as a dictionary to write a SMILES. This dataset has SMILES strings with different character length, this requires an additional preprocessing procedure called padding which involves adding empty

characters (`' '`) to the end of all strings to make sure all the strings has the same length. The length of strings is determined by the longest SMILES string which you'll need to find out.

Task: Encode each character in a SMILES string into categorical numbers (note that this is *not* the same as one-hot encoding) using `LabelEncoder()`. Transform your *encoded* SMILES data into `torch.LongTensor` objects. Create a 60:20:20 split for the train, validation, and test datasets. Next, like what you did in prior assignments, use a `DataLoader` with `batch=512` and `shuffle=True`.

Part 1.2 (15 points) Implement the reparametrization trick

We have provided the implementation for a SMILES-VAE with the `VAE()` class. In VAEs, the encoding of an input x into an embedding z is not deterministic. The model is trying to compress the data into a latent distribution via a conditional distribution $Q_\phi(z|x) = \mathcal{N}(\mu_\phi, \sigma^2_\phi)$ which is parametrized by an encoder function, ϕ. The model needs to sample from the distribution: $z \sim Q_\phi(z|x)$. However, this sampling process requires some extra attention because we want to ensure the sampling procedure is differentiable for gradient optimization. Generally, we cannot do this for a sampling process

$$z \sim \mathcal{N}(\mu_\phi, \sigma^2_\phi) \tag{1}$$

since the derivatives $\frac{dz}{d\mu_\phi}$ and $\frac{dz}{d\sigma_\phi}$ are not clearly defined for this sampling procedure. However, we can use the reparameterization trick which suggests that we randomly sample ϵ from a unit multivariate Gaussian, and then shift the randomly sampled ϵ by the latent distribution's mean μ and scale it by σ.

$$\begin{aligned} \epsilon &\sim \mathcal{N}(0, 1) \\ z &= \mu_\phi + \epsilon \cdot \sigma_\phi \end{aligned} \tag{2}$$

The reparameterization trick makes the sampling process from a multivariate Gaussian differentiable! The reason why it works is because the randomness in the sampling is 'reparameterized' into a leaf node which does not require gradient calculation in the backward computation.[1] To put it more concretely, during the backpropagation process when the gradient $\frac{dL}{dz}$ is computed (L is the scalar loss function), the gradients on μ_ϕ and σ_ϕ can be further computed with the following.

$$\begin{aligned} \frac{dL}{d\mu_\phi} &= \frac{dL}{dz}\frac{dz}{d\mu_\phi} = \frac{dL}{dz} \\ \frac{dL}{d\sigma_\phi} &= \frac{dL}{dz}\frac{dz}{d\sigma_\phi} = \frac{dL}{dz}\epsilon \end{aligned} \tag{3}$$

The distributional output of the encoder input requires additional attention. Note that the encoder module parametrized by an MLP might output a negative σ^2_ϕ which might annoy a statistician. The numerical trick to ensure that σ_ϕ has only positive values is to output the log of σ^2_ϕ instead and then exponentiate:

$$\begin{aligned} \mu_\phi, \log \sigma^2_\phi &= \texttt{encoder(SMILES)} \\ \sigma_\phi &= \exp\left(\frac{1}{2}\log \sigma^2_\phi\right) \end{aligned} \tag{4}$$

[1] The reparameterization trick is a useful technique for variational inference trained with a gradient-based method. Similar reparameterization tricks can be derived for other types of distributions like the Beta, Dirichlet and Von-Mises distributions [2].

Task: Implement the function to transform $\log \sigma_\phi^2$ to σ_ϕ in `VAE.get_std()` (as in equation 4). Then, implement the reparametrization trick in `VAE.reparametrize()` which takes two inputs: μ_ϕ and σ_ϕ and outputs a latent vector z (as in equation 2). You will need the `torch.randn` method to sample ε. Test your code using `VAE.reparametrize()` to generate 1000 samples from a 1D distribution with $\mu = 0$ and $\sigma^2 = 1$ and compare the sampled distribution with $\mathcal{N}(0, 1)$.

Part 1.3 (10 points) Implement the VAE loss function

The decoder model $P_\theta(x|z)$, parameterized by a set of parameters θ, takes the sampled latent variable $z \sim Q_\phi(z|x)$ to reconstruct x which is your input SMILES sequence. The training objective of a VAE is to minimize the negative evidence lower bound, or ELBO, which can be understood as optimizing a reconstruction loss and regularization term. The regularization term is the KL divergence between the parameterized distribution and the prior distribution.

$$\begin{aligned} L &= L_{recon} + \beta L_{regularization} \\ &= \int dz\, Q_\phi(z|x) \log P_\theta(x|z) + \beta \int dz\, p(z) \log \frac{p(z)}{Q_\phi(z|x)} \end{aligned} \tag{5}$$

β is a hyperparameter that balances the two loss terms (see [3] for more information about the effect of β). The reconstruction term compares the original input and the decoded inputs. The input data here is a sequence of vectors with encoded categorical values and the output is a sequence with probabilities for each character categories, so we can use `nn.Functional.cross_entropy()` as the training objective to minimize.

$$L_{recon} = -\frac{1}{N_{seq} N_{char}} \sum_i^{N_{seq}} \sum_k^{N_{char}} p_{data}(\hat{x}_{i,k}) \log(p(x_{i,k})) \tag{6}$$

$\log(p(x_{i,k}))$ is the logit for each possible character category reconstructed by the decoder, $\hat{x}_{i,k}$ is the original SMILES sequence, N_{seq} is the length of the sequence, and N_{char} is the total number of possible characters in the sequence. The `nn.Functional.cross_entropy()` method takes two inputs, the predicted logits for each character at each position in the SMILES sequence (dimension $= N_{batch} \times N_{char} \times N_{seq}$), and the original data as character category represented by integers at each position in the padded SMILES sequence (dimension $= N_{batch} \times N_{seq}$).

A simple prior distribution one can choose is a multivariate Gaussian distributions with all the means as zeros, and all the standard deviations as ones. The parameterized distribution from the encoder is a distribution of the same dimension with parametrized means/standard deviation. Minimizing the Kullback-Leibler (KL) divergence between the parametrized distribution $Q_\phi(z|x)$ and $\mathcal{N}(0, 1)$ encourages the $Q_\phi(z|x)$ to be statistically closer to the Gaussian distribution prior $p(z) = \mathcal{N}(0, 1)$. The KL divergence between the encoded latent distribution (approximated posterior) and the prior has a nice analytical form for Gaussian distributions with a diagonal covariance matrix (you can find the derivation here):

$$\begin{aligned} L_{regularization} &= KL(Q_\phi(z|x)|p(z)) \\ &= KL\left(\mathcal{N}(\mu_\phi(x_i), \sigma_\phi^2(x_i))|\mathcal{N}(0, 1)\right) \\ &= \frac{1}{N_{batch}} \sum_i^{N_{batch}} \frac{1}{2} \left(\sum_d^{N_z} \sigma_{d,\phi}(x_i)^2 + \mu_{d,\phi}(x_i)^2 - \log \sigma_{d,\phi}(x_i)^2 - 1 \right) \end{aligned} \tag{7}$$

$d \in \{1, ..., N_z\}$ is the index for the latent dimension.

Task: For this problem, you need to implement the reconstruction (equation 6) loss and the KL divergence (equation 7) in `loss_function()`. Check your dimensions carefully in this step. In particular, the SMILES input and the decoder output shapes will need the batch size as the first dimension while the second dimension of the decoder output needs to be the number of characters with the last dimensions of both being the sequence length. See the documentation of `nn.Functional.cross_entropy()` for help. Modify your tensor with `transpose()` if necessary to get this to line up.

Part 1.4 (10 points) Train your model

After implementing the reparameterization trick and loss function, you should be able to train a model with the train and test loop we provided to you. We recommend you save the trained model periodically in your Google Drive with the provided code.

Task: Run the provided code chunks to train the VAE for 50 epochs. Make sure you obtain a training and test loss below about 0.15 before proceeding to the next part. Choose `beta=0.001`. This will take around 15 minutes when run on the T4 GPU in Colab.

Part 1.5 (25 points, Grad only) Sample new molecules

The latent space learned by the model is a learned continuous space which you can navigate. The space encodes the complicated molecular 'grammar' of the data it trained on. By sampling vectors z, you can then use a decoder to reconstruct the continuous representation back to a SMILES string (and hopefully molecules). Now use your trained model to sample novel molecules from the learned distribution.

Task 1: Randomly select two SMILES sequences in your test data, encode into latent vectors with the encoder, and then linearly interpolate between the two molecules in the latent space to obtain 10 points in the z space. For each sampled latent code, decode z back to SMILES sequences and test them for validity. You can use the `index2smiles()` function we provided to convert categorical values to SMILES sequences.

Task 2: Next, produce a scatter plot with the first two dimensions of z of your test molecules and newly sampled molecules in the same figure. Color differently these test points and generated points. Among the molecules you generated, how many were decoded into a valid SMILES string? (Don't worry if most of the molecules don't decode into valid SMILES strings.) Use RDKit to show 2D line drawings of valid SMILES you generated. Can you propose a reason for why your VAE sometimes fails to generate valid SMILES strings?

Part 2: Predicting DNA binding sites with transformers

In this part, you will try using transformers to predict DNA binding sites much like you previously did with LSTM models. If you recall from pset 2, you've been provided a dataset of DNA sequences and a binary label 0/1 that indicates if the sequence binds to a protein or not. The sample data format is summarized again in Table 1.

DNA sequence	binder or not
'ATCGGGAA...'	1
'TGCAGTAT...'	0
...	...

Table 1: Dataframe snapshot from DNA binding data

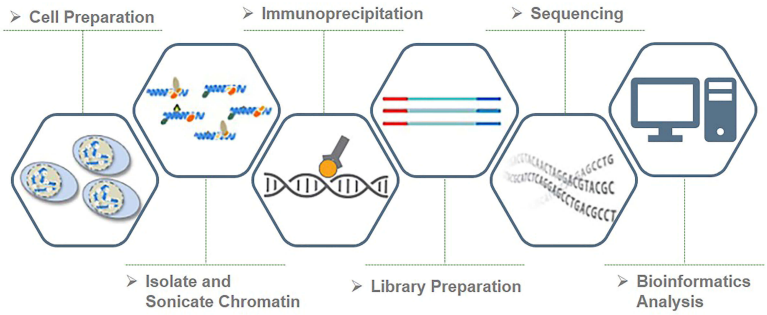

Figure 3: A typical ChIP workflow (source)

Just as was needed for the LSTM model in pset 2, you'll want a GPU for this task. Because the data is formatted as strings like "ATGTCA...", we had one-hot encoded the DNA sequences into bit vectors of size 4 (corresponding to the 4 possible DNA bases A, T, G, and C). We'll reuse much of the code from this previous problem, so you don't have to re-implement the dataset configuration or training/testing functions.

Part 2.1 (15 points) Implement a transformer encoder

Using a transformer encoder, we will attempt to enrich our one-hot encoded sequence using the information contained within the sequence itself. As described in lecture, we need to provide transformers with positional information on how tokens (in our case As, Ts, Gs, and Cs) are arranged in the sequence. The positional encoding module have already been made for you. We'll first need to make the encoder and then define how we're pooling the encoded sequence for predictions.

Task 1: Define your transformer encoder layers (`self.layer`) in `TransformerSeq()` using `nn.TransformerEncoderLayer()`. Set the keyword arguments for this as follows `d_model=d_model`, `dim_feedforward=128`, `nhead=nhead`, and `batch_first=True`. Next, construct the transformer encoder (`self.encoder`) from your layer using `nn.TransformerEncoder()` with arguments `self.layer` and `num_layers` in the respective order (see the documentation for more information).

Task 2: Configure the forward pass of the transformer encoder by passing `x_in` through the encoder you defined above. Because our encoder returns our sequence with the same shape as the input ($N_{\text{batch}} \times N_{\text{seq}} \times N_{\text{d\_model}}$), we need to pool the encodings across the sequence with `torch.mean(x_out, axis=1)` and output the predictions through a `nn.Linear()` and `nn.Sigmoid()`.

Part 2.2 (15 points) Explore how positional encodings improve classification

Now that you've implemented your transformer, we'd like to explore how positional encodings are imperative to learning sequence data. If you look at the transformer implementation, we've added

the `positional` keyword argument. With this setting, we can turn the positional encoding OFF and ON. You won't need to train the transformer for very many epochs, just enough to see the differences.

Task 1: Complete the code to train one `TransformerSeq()` model with positional encodings (`positional=True`) and one without (`positional=False`). For both models, use `d_model=16`, `nhead=8`, and `num_layers=2` and `torch.optim.Adam()` with `lr=1e-3`. Save the training/validation loss for each model with the provided code. Train the models for `100` epochs.

Task 2: Now visualize the training/validation curves between these two models. Make two subplots side-by-side. Comment on what you see. From what you know about transformers, why are positional encodings here necessary? If the two plots happened to be identical, what might that tell us?

References

[1] Gómez-Bombarelli, R. *et al.* Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science* **4**, 268–276 (2018).

[2] Figurnov, M., Mohamed, S. & Mnih, A. Implicit reparameterization gradients. *arXiv preprint arXiv:1805.08498* (2018).

[3] Higgins, I. *et al.* β-VAE: Learning basic visual concepts with a constrained variational framework (2016).